Plotly a Dash

Grafy, PDF report a interaktivní dashboard

Workshop v Pythonu

Společná data

720 syntetických objednávek za 180 dní

Sloupec	Význam
objednavka_id	Jedinečná objednávka
datum	1. 1. až 29. 6. 2025
region	Praha, Brno, Ostrava, Plzeň
kategorie	Knihy, Domácnost, Sport, Elektronika
trzba	Hodnota objednávky v Kč

Jeden řádek = jedna objednávka v jedné kategorii.

Mapa projektu

Stejné označení na snímku i v PyCharmu

`Plotly/`	`Plotly01.py` až `Plotly10.py`
`Dash/`	`Dash01.py` až `Dash12.py`
`data/`	`objednavky.csv`
`vystupy/`	Uložené grafy a PDF report
`prezentace/`	Prezentace a poznámky lektora
`requirements.txt`	Seznam knihoven

- Každý skript funguje samostatně.
- Stejný název najdete na snímku i v projektu.

Spuštění v PyCharmu

Jeden projekt, vybraný interpret Pythonu, jeden běžící server

1. Otevřít celý projekt `workshop_plotly_dash` v PyCharmu.
2. Vybrat Python 3.12 a vlastní virtuální prostředí.
3. Nainstalovat knihovny ve zvoleném prostředí:

```
python -m pip install -r requirements.txt
```

- Spustit `Overeni_prostredi.py`.
- PDF export potřebuje také Chrome nebo Chromium.
- Run spustí příklad; před dalším Dash příkladem použít Stop.

První sloupcový graf

Plotly01.py · Několik hodnot přímo v Pythonu

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
import plotly.express as px

def vytvor_graf():
    kategorie = ["Knihy", "Sport", "Domácnost"]
    trzby = [12_000, 18_000, 15_000]
    fig = px.bar(x=kategorie, y=trzby)
    return fig

if __name__ == "__main__":
    graf = vytvor_graf()
    graf.show(renderer="browser")
```

Seznamy mají stejnou délku. Funkce vrátí objekt grafu (Figure), hlavní blok graf zobrazí. Úplný Plotly01 navíc nastaví nadpis a jednotky os.

Tabulka v Pythonu

Plotly02.py · Nezbytné minimum pandas před Plotly02

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
import pandas as pd
# cesta je v příkladu odvozena od umístění skriptu
data = pd.read_csv(cesta)
print(data.head())
print(data.columns)
print(data["trzba"].sum())
```

- DataFrame je tabulka. Sloupce mají názvy, řádky mají index.
- data[”trzba”] vybere jeden sloupec; data.head() prvních pět řádků.
- V Plotly01 dostalo x seznam. S tabulkou označuje x=”region”její sloupec.

Graf ze souboru CSV

Plotly02.py · Tržby podle regionu

- Souhrnná tabulka má sloupce region a trzba. Názvy v x a y musí odpovídat.

```
fig = px.bar(
    souhrn,
    x="region",
    y="trzba",
    labels={"region": "Region", "trzba": "Tržby (Kč)"},
    title="Tržby podle regionu | syntetická data",
)
```

Před tímto voláním: read_csv načte data a groupby(...)["trzba"].sum() vytvoří souhrn.

Co přesně udělá groupby

Plotly02.py · Na celém CSV: 720 objednávek → 4 součty

```
souhrn = data.groupby(
    "region", as_index=False
)["trzba"].sum()
```

region	trzba
Brno	579 950
Ostrava	366 100
Plzeň	193 770
Praha	738 580

groupby spojí stejné regiony, sum sečte tržby v každé skupině. as_index=False ponechá region jako běžný sloupec pro x.

Co zobrazuje jeden bod

Řádek dat, skupina a souhrnná hodnota

Otázka	**Výpočet**	**Výsledek**
Kolik objednávek?	Počet řádků	720
Jaké celkové tržby?	Součet tržeb	1 878 400 Kč
Jaká průměrná objednávka?	Součet / počet	2 609 Kč

Průměr zobrazujeme zaokrouhlený na celé koruny.

Vývoj v čase

Plotly03.py · Denní tržby

- parse_dates zde převede datum na časové hodnoty. Spojnice spojuje body v pořadí řádků.

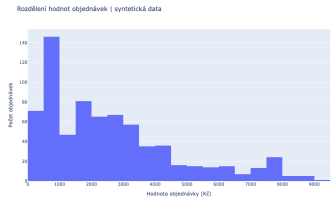
```
data = pd.read_csv(cesta, parse_dates=["datum"])
denne = data.groupby("datum", as_index=False)["trzba"].sum()
denne = denne.sort_values("datum")

fig = px.line(
    denne,
    x="datum",
    y="trzba",
    labels={"datum": "Datum", "trzba": "Denní tržby (Kč)"},
    title="Denní tržby | syntetická data",
)
```

Rozdělení hodnot

Plotly04.py · Histogram hodnot objednávek

- Vstupem jsou jednotlivé objednávky. Výška sloupce je jejich počet.
- nbins=25 žádá nejvýše 25 intervalů; skutečný počet může být menší.




```
fig = px.histogram(
    data,
    x="trzba",
    nbins=25,
    labels={"trzba": "Hodnota objednávky (Kč)"},
    title="Rozdělení hodnot objednávek | syntetická data",
)
```

Co obsahuje objekt grafu (Figure)

Plotly05.py · Data grafu a pravidla jeho zobrazení

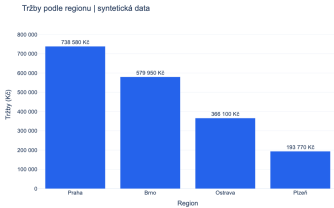
Část objektu grafu	**Co obsahuje**	**Jak ji upravíme**
fig.data	Datové série: hodnoty, typ, vzhled	update_traces(...)
fig.layout	Nadpis, písmo, velikost, okraje	update_layout(...)
Osy v fig.layout	Názvy, rozsahy, značky	update_xaxes / update_yaxes

px.bar nebo px.line vrací objekt Figure. Obsahuje data i nastavení grafu; show jej zobrazí, write_html a write_image uloží. Název proměnné fig je naše zkratka.

Čitelný graf

Plotly05.py · Popisky, jednotky, barvy a text při najetí myší

- Vzhled série mění update_traces. Nadpis a velikost patří do update_layout.
- Popisek při najetí myší (hover): %{x} a %{y} se doplní z bodu; <extra></extra> skryje vedlejší rámeček s názvem série.



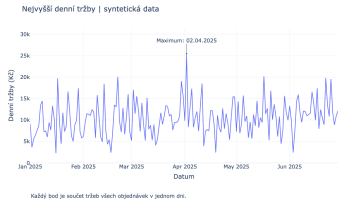

```
fig.update_traces(
    marker_color="#2563eb",
    texttemplate="%{y:,.0f} Kč",
    textposition="outside",
    cliponaxis=False,
    hovertemplate="<b>%{x}</b><br>Tržby: %{y:,.0f} Kč<extra></extra>",
)
```

Text přímo v grafu

Plotly06.py · Komentář k významnému místu

- Anotace je text doplněný do grafu. Zde x a y určují datum a tržbu; šipka označí maximum.

```
fig.add_annotation(
    x=maximum["datum"],
    y=maximum["trzba"],
    xref="x",
    yref="y",
    text=f"Maximum: {maximum['datum']:%d.%m.%Y}",
    showarrow=True,
    arrowhead=2,
    ax=0,
    ay=-45,
    bgcolor="white",
)
```



Text mimo datový bod

Plotly06.py · Souřadnice paper a místo pro komentář

```
fig.add_annotation(
    x=0,
    y=-0.27,
    xref="paper",
    yref="paper",
    xanchor="left",
    text="Každý bod je součet tržeb všech objednávek v jednom dni.",
    showarrow=False,
)
```

- paper: (0, 0) je levý dolní a (1, 1) pravý horní roh plochy bez okrajů.
- Záporné y umístí text pod graf; margin={"b": 140} mu rezervuje místo.
- xanchor/yanchor určí ukotvení textového pole.
 zalomí řádek, <b> zvýrazní text.

Objekt grafu a datové série

Plotly07.py · Co skládáme do společného dashboardu

- Datová série (trace) popisuje jednu sadu sloupců, bodů nebo spojnic.
- Jeden objekt Figure může obsahovat více sérií i více panelů s grafy.
- fig.data obsahuje série. Cyklus je přidá do cílového objektu; nadpis ani nastavení os se tím nepřenášejí.

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
casovy_graf = px.line(denne, x="datum", y="trzba")
fig = make_subplots(rows=1, cols=2)
for serie in casovy_graf.data:
    fig.add_trace(serie, row=1, col=1)
```

O1: porovnání regionů a kategorií

Průběžné ověření Plotly · 20–25 minut · Plotly_Pru01.py

1. Načtěte data/objednavky.csv. Pro každou dvojici region–kategorie spočítejte součet tržeb a počet objednávek.
2. V jednom grafu zobrazte sloupce kategorií vedle sebe. Regiony: Praha, Brno, Ostrava, Plzeň. Kategorie rozlište barvou a legendou.
3. Při najetí myší ukažte tržbu v Kč a počet objednávek dané skupiny.
4. Přidejte přerušovanou čáru ve výši 100 000 Kč; označte ji jako zvolenou srovnávací hranici. Do komentáře napište celkové tržby.

Dohledejte: sdružené sloupce a pořadí kategorií, custom_data a hovertemplate, add_hline; pandas: více souhrnů pomocí agg.

Kontrola: 4 série, součet 1 878 400 Kč. Plzeň nemá objednávky elektroniky; sloupec této kombinace chybí.

O1: vybrané části řešení

Souhrn dat, doplňková hodnota u sloupce a srovnávací čára

Zkrácené úryvky; proměnná data označuje načtenou tabulku objednávek.

- agg vytvoří součet i počet; px.bar samo objednávky nesčítá.
- custom_data má v px podtržítko, ale customdata v popisku ne. Index 0 vybírá první předaný sloupec.
- add_hline přidá tvar do nastavení grafu, nikoli pátou datovou sérii.

Dva grafy vedle sebe

Plotly07.py · První společná kompozice

- make_subplots vytvoří jeden objekt Figure s více panely (oblastmi pro grafy). Pozice row a col číslujeme od 1.

```
fig = make_subplots(
    rows=1,
    cols=2,
    subplot_titles=("Denní tržby", "Tržby podle regionu"),
    horizontal_spacing=0.13,
)

for serie in casovy_graf.data:
    fig.add_trace(serie, row=1, col=1)
for serie in regionalni_graf.data:
    fig.add_trace(serie, row=1, col=2)
```

Nejdříve navrhneme mřížku

Plotly08.py · Dva řádky a dva sloupce v Plotly08

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
fig = make_subplots(
    rows=2, cols=2,
    specs=[
        [{"colspan": 2}, None],
        [{}, None],
    ],
)
```

Pozice	Obsah
(1, 1) přes 2 sloupce	Časový graf
(2, 1)	Sloupcový graf
(2, 2)	Místo pro text bez os

- Každý vnitřní seznam je jeden řádek mřížky. {} vytvoří panel.
- colspan=2 panel rozšíří; následující None označí obsazené místo.
- None vpravo dole ponechá prázdné místo. Text vložíme anotací.

Rozvržení stránky

Plotly08.py · Grafy a text mají společný prostor

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
fig.add_annotation(
    x=0.59, y=0.30, xref="paper", yref="paper",
    xanchor="left", yanchor="top", align="left",
    text=("<b>Jak číst tento přehled</b><br>"
          "Horní graf ukazuje vývoj po dnech.<br>"
          "Dolní graf porovnává regiony."),
    showarrow=False,
    font={"size": 19},
)
```

- Anotace vyplní prázdnou pravou dolní část mřížky.
- Poloha ukotvuje levý horní roh textu; text roste doprava a dolů.
- Úplný příklad má delší komentář a také nastavuje osy, velikost a okraje.

HTML, PNG a PDF

Stejný objekt grafu, různé způsoby předání

Formát	**Co zachová**	**Typické použití**
HTML	Popisky při najetí, přiblížení a ovládání legendy	Prohlížení v prohlížeči
PNG	Statický rastrový obrázek	Vložení do dokumentu
PDF	Statický výstup	Sdílení a tisk reportu

PNG a PDF vykreslí Kaleido pomocí Chrome.

Uložení výsledku

Plotly09.py · Soubory ve složce vystupy

- HTML obsahuje Plotly.js a zachová interaktivní popisky. PNG a PDF vykreslí Kaleido s Chrome.

Zjednodušený zápis; úplný kód je v navazujícím souboru.

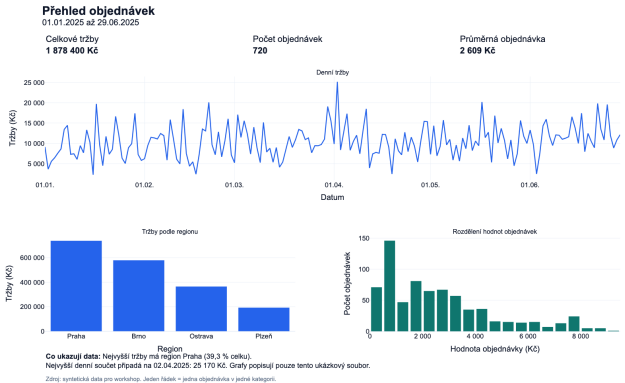
```
slozka = Path(__file__).resolve().parents[1] / "vystupy"
slozka.mkdir(exist_ok=True)
fig.write_html(slozka / "Plotly09.html", include_plotlyjs=True)
fig.write_image(slozka / "Plotly09.png", scale=2)
fig.write_image(slozka / "Plotly09.pdf")
```

Zkrácený zápis stejného exportu. Originální skript navíc vypíše cesty. PDF má jednu stránku danou rozvržením objektu grafu; nejde o automatickou sazbu dokumentu.

Hotový PDF dashboard

Plotly10.py · První dokončený výstup

- Více grafů na jedné stránce
- Období, souhrn a komentář
- Export přímo z Plotly



Z čeho vznikne finální PDF

Plotly10.py · Šest označených částí funkce vytvor_graf v Plotly10

- 1. Data a souhrny.
- 2. Mřížka se spojenými buňkami.
- 3. Grafy a popisky os.
- 4. Ukazatele jako textové anotace.
- 5. Komentář a zdroj dat.
- 6. Nadpis, období a okraje.

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
celkove_trzby = data["trzba"].sum()
hodnota = format_kc(celkove_trzby)
fig.add_annotation(
    x=0, y=1.08,
    xref="paper", yref="paper",
    xanchor="left", yanchor="bottom",
    text=f"Tržby<br><b>{hodnota}</b>",
    showarrow=False,
)
```

Takto vložíme jeden vypočtený ukazatel. V Plotly10 smyčka vytvoří tři; okraj t nad grafy ponechá místo pro y > 1. Potom uloz_vystupy provede export.

Funkce pro tvorbu grafu

Plotly10.py · Oddělíme výpočty, tvorbu grafu a export

- Funkce přijme data a vrátí objekt grafu. Načítání, zobrazení a export řídí okolní program.

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
def vytvor_graf(data):
    souhrn = data.groupby("region", as_index=False)["trzba"].sum()
    fig = px.bar(souhrn, x="region", y="trzba")
    return fig

# Stejný princip jako v Plotly10, zde pouze jeden graf.
data = nacti_data()
vyber = data.loc[data["region"] == "Brno"]
graf = vytvor_graf(vyber)
graf.show(renderer="browser")
```

Úplná verze Plotly10 vrací celý dashboard a při prázdném vstupu vyvolá srozumitelnou ValueError.

C1: vlastní PDF po Plotly

30 minut samostatné práce v PyCharmu

- Vytvořte Plotly/Plotly11_Cukrarna.py a jeden objekt Figure se dvěma grafy.
- Graf 1: náklad na jeden kus pro všech šest výrobků.
- Graf 2: modelový zisk na jeden kus včetně záporné makronky.
- Přidejte nadpis, jednotky Kč/ks a krátký komentář vypočtený z dat.
- Uložte vystupy/cukrarna_aktivita.pdf a otevřete celou stránku.

Použijte připravené kalkulace a známé kroky z Plotly07–10. Vlastní rozvržení a barvy mohou být jiné.

Data pro samostatné PDF

kalkulace.csv: jeden řádek představuje jeden výrobek

```
from pathlib import Path
import pandas as pd

cesta = (Path(__file__).resolve().parents[1]
         / "data" / "cukrarna" / "kalkulace.csv")
data = pd.read_csv(cesta)
print(data[["nazev", "naklady_ks_kc", "zisk_ks_kc"]])
```

- nazev: popisek výrobku; naklady_ks_kc a zisk_ks_kc: hodnoty obou grafů.
- Tabulka už obsahuje kalkulaci. Znovu ji nesčítejte podle objednávek.
- Jde o snímek výpočtu ze tří zdrojových CSV; změna ceníku jej sama neaktualizuje.

Kalkulace jedné dávky a jednoho kusu

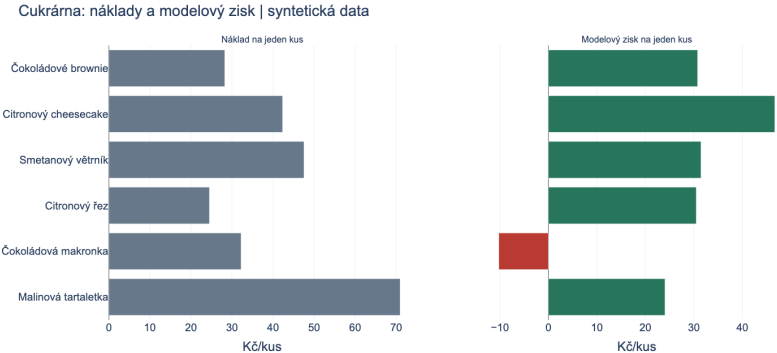
Sazba práce v modelu: 240 Kč za hodinu

Krok	Výpočet	Brownie
Spotřeba másla	200 / 250 × 60	48,00 Kč
Všechny suroviny	Součet položek celé dávky	162,76 Kč
Práce	35 / 60 × 240	140,00 Kč
Celá dávka	Suroviny + práce + ostatní 36 Kč	338,76 Kč
Náklad jednoho kusu	338,76 / 12 kusů	28,23 Kč
Modelový zisk na kus	Cena 59 − náklad 28,23	30,77 Kč

Marže (%) = zisk / prodejní cena × 100. Model nezahrnuje fixní režii, DPH ani daně. Zaokrouhlujeme až zobrazení.

C1: příklad hotového PDF

Dva grafy a komentář v jednom objektu Figure



C1: kontrola před přechodem k Dash

Výsledek otevřete a porovnejte s daty

Výrobek	**Náklad / ks**	**Modelový zisk / ks**
Čokoládové brownie	28,23 Kč	30,77 Kč
Čokoládová makronka	32,21 Kč	−10,21 Kč

- Jedno PDF, dva grafy, v každém všech šest výrobků.
- Osy a text jsou čitelné i ve statickém PDF; nic se nepřekrývá.
- Komentář označuje smyšlená data a omezení nákladového modelu.
- Na jednom výrobku vysvětlete: cena − náklad = modelový zisk.

Volitelné rozšíření PDF

Podrobný report po dokončení základní aktivity

- Jedna stránka: nadpis, tři ukazatele a označení syntetických dat.
- Porovnání prodejní ceny s celkovým nákladem na jeden kus.
- Rozpad nákladu na suroviny, práci a ostatní náklady.
- Přehled šesti výrobků: cena, náklad, modelový zisk a marže.
- Komentář vypočtený z dat: nejvyšší modelový zisk na kus a ztrátový výrobek.

Navazujte na Plotly07–10. Přehled může být tabulka go.Table nebo další čitelný graf. Uložte PDF a zkontrolujte celou stránku.

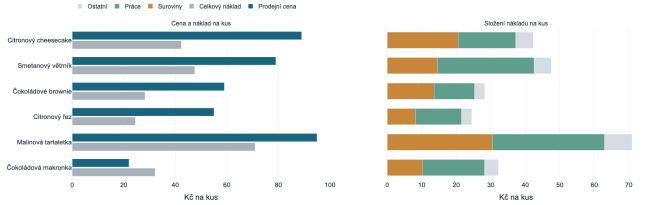
Vzor rozšířeného PDF reportu

Volitelné: ukazatele, rozpad nákladů a tabulka

Cukrárna: kalkulace výrobků

Syntetická data | sazba práce 240 Kč/h

Počet receptů	Nejvyšší modelový zisk / ks	Ztrátových výrobků
6	**46,69 Kč**	**1**



Kalkulace výrobků

Výrobek	Ks / dávka	Cena / ks	Náklad / ks	Modelový zisk / ks	Marže
Citronový cheesecake	12	89,00 Kč	42,31 Kč	46,69 Kč	52,5 %
Smetanový věterník	10	79,00 Kč	47,53 Kč	31,47 Kč	39,8 %
Čokoládové brownie	12	59,00 Kč	28,23 Kč	30,77 Kč	52,2 %
Citronový řez	12	55,00 Kč	24,50 Kč	30,50 Kč	55,5 %
Malinová tartaletka	8	95,00 Kč	70,95 Kč	24,05 Kč	25,3 %
Čokoládová makronka	20	22,00 Kč	32,21 Kč	-10,21 Kč	-46,4 %

Výsledek: Nejvyšší modelový zisk na kus má Citronový cheesecake: 46,69 Kč.
Ztrátové výrobky: Čokoládová makronka. Jejich cena nepokrývá započtené náklady.

Modelový zisk = cena − suroviny − práce − ostatní zadané náklady. Firmí režii, DPH ani daně nepočítáme.
Zdroj: smyšlené výukové ceny a recepty. Bez údajů o prodaném množství nepočítáme celkový zisk cukrárny.

Webový dashboard v Dash

Dashboard = přehled s grafy, ukazateli a ovládáním

Pojem	**Co určuje**
Plotly fig.layout	Nadpis, osy, anotace a vzhled objektu grafu.
Dash app.layout	Strom webových komponent: texty, ovladače a grafy.
Dash callback	Obslužnou funkci; zde běží v Pythonu na serveru a reaguje na vstup.

Popisky při najetí myší a přibližování fungují uvnitř grafu Plotly. Callback může navíc přepočítat data podle vstupu uživatele.

První spuštění Dash

Dash01.py · Prázdný obsah jako nejmenší aplikace

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
from dash import Dash, html

app = Dash(__name__)
app.layout = html.Div()

if __name__ == "__main__":
    app.run(debug=True, use_reloader=False)
```

Po Run otevřete adresu z konzole, obvykle http://127.0.0.1:8050/. Prázdný obsah aplikace je správně; může být vidět vývojový panel Dash.

Python a prohlížeč jsou dvě části

Dash01.py · Provoz aplikace v PyCharmu

Krok	**Co se děje**
1. Run v PyCharmu	Python spustí server; konzole zůstane aktivní.
2. Otevřít 127.0.0.1:8050	Prohlížeč zobrazí stránku místního serveru.
3. Změnit kód	Příklady používají use_reloader=False.
4. Stop → Run → obnovit stránku	Nový proces načte upravený kód.

Zavření karty server neukončí. Před dalším příkladem zastavte předchozí server, jinak bude port 8050 obsazený.

Nadpis a text

Dash02.py · Obsah stránky tvoří komponenty

- children určuje obsah; zde je to seznam komponent v pořadí zobrazení.

```
app.layout = html.Div(children=[
    html.H1("Přehled objednávek"),
    html.P(
        "Jak se vyvíjejí tržby a jaké "
        "rozdíly vidíme mezi regiony?"
    ),
    html.P("Pracujeme se syntetickými daty vytvořenými pro workshop."),
])
```

Strom komponent a jejich vlastnosti

Dash02.py · Slovník pro čtení dalších příkladů

Vlastnost	**Význam**	**Příklad**
children	Obsah: text nebo další komponenty	html.Div(children=[...])
id	Jedinečný identifikátor v aplikaci	id="region"
value	Aktuální hodnota ovladače	value="Všechny"
figure	Objekt grafu pro dcc.Graph	figure=graf
style	Vzhled webové komponenty	style={"padding": "20px"}

app.layout → Div → [H1, P, P]. Nadpis a odstavce jsou vnořené komponenty uvnitř Div. Seznam určuje jejich pořadí.

První graf v aplikaci

Dash03.py · Objekt grafu Plotly jako obsah dcc.Graph

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
from dash import Dash, dcc, html

app.layout = html.Div([
    html.H1("Tržby podle regionu"),
    dcc.Graph(id="graf-region", figure=vytvor_graf_regionu(df)),
])
```

- Funkce vytvor_graf_regionu(df) vytvoří objekt grafu pro dcc.Graph.
- V úplném souboru před definicí app.layout najdete data, definici funkce a app = Dash(...).
- Titulek a barvy uvnitř grafu měníme přes Plotly; umístění komponenty přes Dash.

Dva grafy na stránce

Dash04.py · Rozvržení obsahu v Dash

- Dvě komponenty dcc.Graph rozmístí html.Div. Každá má vlastní objekt Figure.

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
app.layout = html.Div([
    html.H1("Přehled objednávek"),
    html.Div([
        dcc.Graph(id="graf-cas", figure=vytvor_graf_casu(df)),
        dcc.Graph(id="graf-region", figure=vytvor_graf_regionu(df)),
    ], style={"display": "flex", "flexWrap": "wrap", "gap": "20px"}),
])
```

display=”flex”řadí vnořené komponenty vedle sebe, flexWrap=”wrap”dovolí přesun pod sebe a gap určuje mezeru.

Kartičky s ukazateli

Dash05.py · Tržby, počet a průměrná objednávka

```
trzby = df["trzba"].sum()
pocet = len(df)
prumer = df["trzba"].mean()
```

Částky formátuje format_kc na text s Kč.
Počet převedeme na text pomocí str.
Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
vytvor_kartu("Tržby", format_kc(trzby), "kpi-trzby")
```

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
def vytvor_kartu(nazev, hodnota,
                 id_hodnoty):
    return html.Div([
        html.P(nazev),
        html.H2(hodnota, id=id_hodnoty),
    ])
```

Vrácený Div vložíme do seznamu v app.layout. Plná funkce navíc nastavuje vzhled karty.

O2: vlastní přehled kategorií

Průběžné ověření Dash · 15–20 minut · Dash_Pru02.py

1. Uložte kopii Dash05.py jako Dash_Pru02.py. Nad všemi objednávkami vytvořte stránku s nadpisem a obdobím dat.
2. Pomocí jedné funkce vytvořte čtyři karty: tržby, počet objednávek, průměrnou a největší objednávku.
3. Přidejte sloupce tržeb podle kategorií a histogram hodnot objednávek. K prvnímu grafu potřebujete souhrn, ke druhému jednotlivé řádky.
4. Pod grafy vložte rozbalovací text „Jak číst přehled“. Vysvětlete význam jednoho řádku dat a výšku sloupců histogramu.

Dohledejte: html.Details a html.Summary. Pro změnu velikosti grafu: dcc.Graph(responsive=True).

Kontrola: 720 objednávek, tržby 1 878 400 Kč. Zužte okno: karty a grafy se mají přeskládat pod sebe. Callback zatím nepotřebujete.

O2: vybrané části řešení

Výpočet karty, změna velikosti grafu a rozbalovací text

Výřezy z programu; komponenty vložíme do seznamu children v app.layout.

- max počítejte z jednotlivých objednávek; nejvyšší součet kategorie je jiný ukazatel.
- responsive přizpůsobí graf prostoru. Přeskládání komponent řeší obalový Div: display=”flex”, flexWrap=”wrap”.
- Summary je klikací záhlaví Details. Otevření obslouží prohlížeč bez Python callbacku.

Výběr regionu

Dash06.py · Ovladač zatím není propojený

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
regiony = ["Všechny"] + sorted(df["region"].unique())
dcc.Dropdown(
    id="region", options=regiony,
    value="Všechny", clearable=False,
)
```

- unique vybere různé regiony. sorted je seřadí. Všechny přidáváme sami.
- options jsou možnosti, value aktuální hodnota a id název pro callback.
- clearable=False brání uživateli vymazat výběr. Dropdown zatím mění pouze sebe.

První callback

Dash07.py · Vybraný region se objeví v textu

from dash import Dash, Input, Output, dcc, html

```
@app.callback(
    Output("popis-vyberu", "children"),
    Input("region", "value"),
)
def zobraz_vyber(region):
    if region == "Všechny":
        return "Vybrané regiony: všechny."
    return f"Vybraný region: {region}."
```

Output určuje cíl, Input zdroj. Parametr region dostane vybraný řetězec.

Propojení musí sedět na obou stranách

Dash07.py · Stejná id v komponentách stránky a v callbacku

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
app.layout = html.Div([
    dcc.Dropdown(
        id="region", options=regiony,
        value="Všechny", clearable=False),
    html.P(id="popis-vyberu"),
])
```

```
@app.callback(
    Output("popis-vyberu", "children"),
    Input("region", "value"),
)
def zobraz_vyber(region):
    if region == "Všechny":
        return "Vybrané regiony: všechny."
    return f"Vybraný region: {region}."
```

Input dodá hodnotu parametru region. return nahradí obsah odstavce; funkci vyvolává Dash také při prvním načtení.

Jak proběhne aktualizace

Dash07.py · Jedna změna vstupu spustí známou funkci

- Při načtení stránky Dash zavolá callback s výchozí hodnotou Všechny.
- Po změně region.value zavolá funkci znovu s aktuálním výběrem.
- Výsledek return nahradí popis-vyberu.children.
- Pro graf budeme vracet objekt grafu do vlastnosti figure.

Funkci nevoláme v hlavním bloku ručně. Vazbu registruje dekorátor @app.callback nad její definicí.

Filtr mění graf

Dash08.py · Vybraná data se znovu vykreslí

```
@app.callback(
    Output("graf-cas", "figure"),
    Input("region", "value"),
)
def aktualizuj_graf(region):
    if region == "Všechny":
        vyber = df
    else:
        maska = df["region"] == region
        vyber = df.loc[maska]
    return vytvor_graf_casu(vyber)
```

vytvor_graf_casu(vyber) uvnitř sečte tržby po dnech a vrátí graf vytvořený voláním px.line. Callback aktualizuje graf-cas.figure.

Maska vybere řádky před agregací

Dash08.py · Všechny znamená návrat k původní tabulce

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
maska = df["region"] == "Praha"
vyber = df.loc[maska]
souhrn = vyber.groupby("datum", as_index=False)["trzba"].sum()
fig = px.line(souhrn, x="datum", y="trzba")
```

- Porovnání vrátí pro každý řádek True nebo False. loc ponechá True.
- Nejprve filtrujeme objednávky, potom počítáme součty z vybraných řádků.
- Při volbě Všechny použijeme celé df. Globální df nepřepisujeme filtrem.

Jeden filtr, více výstupů

Dash09.py · Grafy i karty ukazují stejný výběr

```
@app.callback(
    Output("graf-cas", "figure"),
    Output("graf-region", "figure"),
    Output("kpi-trzby", "children"),
    Output("kpi-pocet", "children"),
    Output("kpi-prumer", "children"),
    Output("popis-vyberu", "children"),
    Input("region", "value"),
)
def aktualizuj_dashboard(region):
```

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
return (
    vytvor_graf_casu(vyber),
    vytvor_graf_regionu(vyber),
    trzby,
    pocet,
    prumer,
    popis,
)
```

Mezi definicí a return se vybere tabulka a vypočtou trzby, pocet, prumer a popis. Úplná funkce je v Dash09.py; pořadí musí zůstat shodné.

Šest výstupů, šest hodnot

Dash09.py · Rozšíření callbacku znamená změnu komponent i návratových hodnot

Pořadí	**Output**	**Odpovídající hodnota return**
1	graf-cas.figure	vytvor_graf_casu(vyber)
2	graf-region.figure	vytvor_graf_regionu(vyber)
3	kpi-trzby.children	trzby
4	kpi-pocet.children	pocet
5	kpi-prumer.children	prumer
6	popis-vyberu.children	popis

Všechny hodnoty vycházejí ze stejného vyber. Pro další kartu potřebujeme nové id v app.layout, Output i sedmou návratovou hodnotu.

Dva současné filtry

Dash10.py · Region a kategorie se kombinují

```
def vyber_data(region, kategorie):
    vyber = df
    if region != "Všechny":
        vyber = vyber.loc[vyber["region"] == region]
    if kategorie != "Všechny":
        vyber = vyber.loc[vyber["kategorie"] == kategorie]
    return vyber
```

Dva vstupy Input v dekorátoru: region.value a kategorie.value. Jejich pořadí odpovídá parametrům aktualizuj_dashboard(region, kategorie).

Druhé filtrování zužuje první výběr: logické AND. Kombinace Plzeň + Elektronika vrátí nula řádků.

Prázdný výběr

Dash10.py · Nula objednávek má jasné zobrazení

```
def vytvor_graf_casu(data):
    if data.empty:
        return prazdny_graf("Denní tržby")
    souhrn = data.groupby("datum", as_index=False)["trzba"].sum()
    return px.line(souhrn, x="datum", y="trzba", title="Denní tržby",
                   labels={"datum": "Datum", "trzba": "Tržby (Kč)"})
```

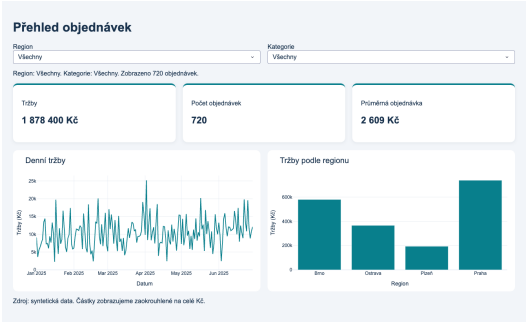
```
if vyber.empty:
    prumer = "-"
else:
    prumer = format_kc(vyber["trzba"].mean())
```

Součet a počet jsou 0. prazdny_graf vrátí nový objekt grafu se zprávou a skrytými osami, aby nezůstal předchozí graf.

Vzhled dashboardu

Dash11.py · Stejný obsah v čitelném rozvržení

Styl	Význam
backgroundColor	Barva pozadí
padding	Vnitřní odsazení
gap	Mezera mezi komponentami
fontSize	Velikost písma
borderRadius	Zaoblení rohů

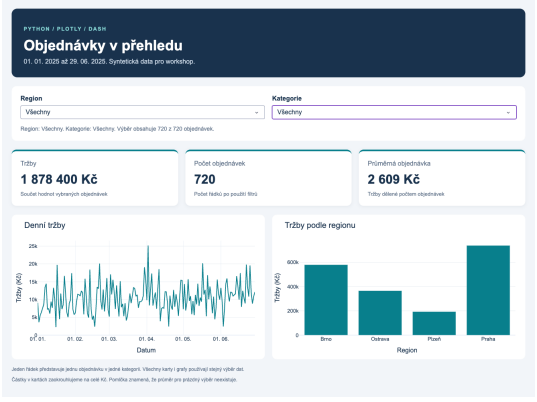


Slovníky STYL_RADKU a STYL_KARTY jsou připravené v Dash11. Upravte jednu hodnotu a ověřte čitelnost i úzké okno.

Hotový webový dashboard

Dash12.py · Druhý dokončený výstup

- Tři karty a dva propojené grafy
- Společné filtry a popis výběru
- Přehledné funkce pro data a zobrazení



Dny bez objednávek v časovém grafu

Dash12.py · Nový datový krok v Dash12 oproti Dash11

```
vsechny_dny = pd.date_range(df["datum"].min(), df["datum"].max(), name="datum")
souhrn = data.groupby("datum")["trzba"].sum()
souhrn = souhrn.reindex(vsechny_dny, fill_value=0)
souhrn = souhrn.rename("trzba").reset_index()
```

- date_range vytvoří všechny dny období podle původního df.
- groupby počítá jen z vybraných objednávek. Datum je zde index.
- reindex doplní chybějící dny nulou; reset_index vrátí datum do sloupce.

Nula je správně, protože máme úplný soubor objednávek. Chybějící měření může znamenat neznámou hodnotu; tam nulu automaticky nedoplňujeme.

P5: samostatná práce po Dash

Dash12.py · Jedna změna, kterou dokážete vysvětlit

- P5 · 25 minut: samostatně změňte funkci dashboardu a vysvětlete řešení.
- Doporučené zadání: regionální graf v Dash12 ukáže počet objednávek.
- Upravte agregaci, hodnotu y, název osy, nadpis i popisek při najetí myší.
- Zachovejte filtry a prázdný stav. Karty dál zobrazují tržby, počet a průměr.

CVICENI.md obsahuje celé náhradní tělo funkce i kontrolní hodnoty. Řešení odkryjte až po vlastním pokusu.

C2: rozšíření po části Dash

Receptář se dvěma záložkami; samostatný navazující blok

- Po dokončení P5 můžete přejít k nové aplikaci nad cukrárnou.
- Nabídka a zisk: kategorie, karty a porovnání výrobků.
- Receptář: výběr výrobku, suroviny, postup a přepočet na více dávek.
- Zde už načtete tři zdrojové tabulky a sami spojíte ceník s recepty.

Celá C2 potřebuje přibližně 50–60 minut práce navíc. Patří do pokračování nebo domů. Nová průběžná ověření prodlužují původní plán o dalších 35–45 minut.

Tři tabulky cukrárny

Soubory ve složce data/cukrarna

Soubor	Jeden řádek znamená	Propojení
suroviny.csv	Surovina: jednotka, velikost a cena balení.	surovina_id
recepty.csv	Recept a normovaná dávka: počet kusů, cena za kus, práce a ostatní náklady.	recept_id
polozky_receptu.csv	Množství jedné suroviny v jedné dávce.	Oba identifikátory

Množství i balení používají stejnou jednotku: g, ml nebo ks. Cena balení není cenou spotřebované suroviny v receptu.

Spojení ceníku s položkami receptu

Nový nástroj pandas: merge podle společného identifikátoru

```
detail = polozky.merge(
    suroviny, on="surovina_id", how="left",
    validate="many_to_one",
)
detail["naklad_polozky_kc"] = (
    detail["mnozstvi"] / detail["velikost_baleni"]
    * detail["cena_baleni_kc"]
)
souhrn = detail.groupby("recept_id", as_index=False)["naklad_polozky_kc"].sum()
```

- merge připojí údaje suroviny ke každé položce; left zachová všechny položky.
- many_to_one ověří, že ceník má pro každé id nejvýše jeden řádek.
- Součet po recept_id připojte k receptům a dopočítejte práci, náklad a zisk.

C2: aplikace se dvěma záložkami

Soubor, který vytvoříte: Dash13_Cukrarna.py

- Nabídka a zisk
- Filtr kategorie včetně Všechny.
- Karty: počet výrobků, nejvyšší zisk na kus, počet ztrátových.
- Graf ceny a nákladu, graf zisku na kus.
- Receptář
- Výběr receptu a počtu dávek (1–3).
- Karty: kusy, náklad a modelový zisk dávek při prodeji všech kusů.
- Tabulka surovin, postup a graf nákladů surovin.

Obě záložky používají stejná data. Filtr kategorie patří pouze nabídce; výběr receptu a dávky patří receptáři.

Jak vytvořit dvě záložky

Obsah předáme přímo jako children jednotlivých Tab

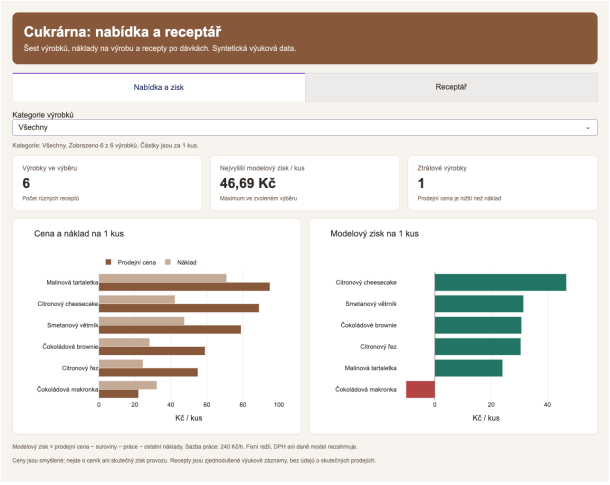
```
from dash import Dash, dcc, html

app = Dash(__name__)
app.layout = dcc.Tabs(value="nabidka", children=[
    dcc.Tab(label="Nabídka a zisk", value="nabidka", children=[
        html.P("Sem přijde přehled výrobků a jeho ovládání."),
    ]),
    dcc.Tab(label="Receptář", value="receptar", children=[
        html.P("Sem přijde výběr receptu a jeho podrobnosti."),
    ]),
])
```

Toto je pouze kostra rozvržení stránky (app.layout). Přepnutí záložky obslouží dcc.Tabs; známé callbacky budou přepočítávat obsah podle ovladačů.

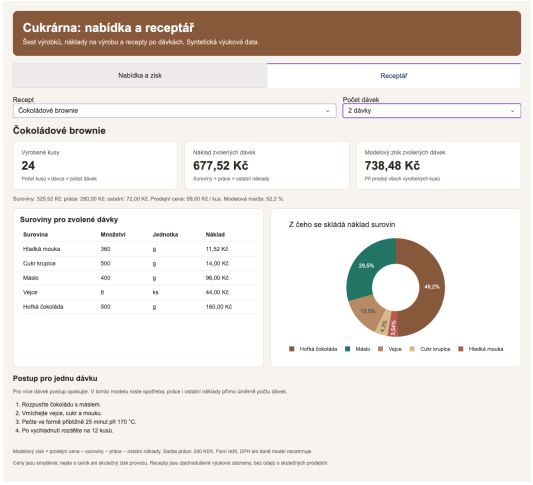
C2: záložka Nabídka a zisk

Vzor s vybranými všemi kategoriemi



C2: záložka Receptář

Vzor: brownie ve dvou dávkách



C2: kontrola receptáře a kalkulace

Výpočet na kus a na zvolený počet dávek

Scénář	Počet kusů	Náklad celkem	Modelový zisk celkem
Brownie: 1 dávka	12	338,76 Kč	369,24 Kč
Brownie: 2 dávky	24	677,52 Kč	738,48 Kč
Makronka: 1 dávka	20	644,18 Kč	−204,18 Kč

- Nabídka: Všechny → Řezy → Drobné pečivo → Všechny.
- Receptář: změnit výrobek, změnit dávky a vrátit původní výběr.
- Modelový zisk dávky předpokládá prodej všech vyrobených kusů.
- Předvedete výsledek a vysvětlíte propojení dat, výpočet a jednu kontrolu.

Kontrola obou výstupů

C1: cukrárna v PDF; P5: aplikace nad objednávkami

Výběr	Tržby	Počet	Průměr
Všechny + Všechny	1 878 400 Kč	720	2 609 Kč
Praha + Knihy	47 210 Kč	73	647 Kč
Plzeň + Elektronika	0 Kč	0	—

- C1: dva grafy cukrárny, jednotky, komentář, zdroj a jedna čitelná stránka PDF.
- Dash: oba grafy i všechny karty reagují; návrat na Všechny obnoví čísla.
- Každý finální skript spustíme samostatně z PyCharmu.

Použití na vlastních datech

Známý postup pro další projekt

- Nejdříve určete, co znamená jeden řádek a jaký ukazatel odpovídá otázce.
- Upravte názvy sloupců, datové typy, jednotky a možnosti filtrů.
- Prověřte chybějící hodnoty a období; nulou nedoplňujte neznámá měření.
- Plotly10 očekává neprázdná data a kladný součet tržeb jako v ukázkovém CSV.

Odevzdávaný projekt obsahuje zdroje, data, řešení a prezentaci. Virtuální prostředí si vytvoří každý na svém počítači.

Příloha: slovník pojmů

Referenční snímek mimo hlavní výklad

Pojem	**Význam v tomto workshopu**
DataFrame / Series	Tabulka / jeden sloupec nebo řada hodnot.
Figure / trace	Objekt grafu (i s více panely) / jedna datová série.
Agregace	Více objednávek shrneme součtem nebo počtem.
Komponenta	Prvek stránky: text, ovladač nebo graf.
Callback	Funkce volaná Dash při změně vstupu, zde i při načtení.
KPI	Klíčový ukazatel výkonnosti; zde zobrazený na kartě.

Příloha: čtyři recepty pandas

Čtěte vstup, operaci a výstup

Zjednodušený zápis; úplný kód je v navazujícím souboru.

```
# Součet za regiony: tabulka se sloupci region, trzba.
souhrn = data.groupby("region", as_index=False)["trzba"].sum()
# Počet řádků za regiony: tabulka se sloupci region, pocet.
pocty = data.groupby("region").size().reset_index(name="pocet")
# Jedna kategorie: výběr původních řádků.
vyber = data.loc[data["kategorie"] == "Knihy"]
# Jedna hodnota: medián objednávek ve výběru.
median = vyber["trzba"].median()
```

Příloha: když se výsledek nezmění

Krátký postup hledání příčiny

Projev	**Co zkontrolovat**
Nejde import knihovny	Vybraný interpret Pythonu a instalaci přes jeho python -m pip.
PDF export selže	Overeni_prostredi.py, Kaleido a Chrome/Chromium.
Adresa nejde otevřít	Běží správný server? Jakou adresu vypsala konzole?
Port 8050 je obsazený	Zastavit předchozí vlastní Dash příklad.
Stále vidím starý výsledek	Stop → Run → obnovení stránky; otevřen správný soubor.
Callback hlásí chybu	Konzole; shoda id, vlastností a počtu Output/return.

Příloha: kontrola vlastního CSV

Před připojením nových dat

- Existují potřebné sloupce a znamená jeden řádek opravdu jednu objednávku?
- Je datum načtené jako datum a tržba jako číslo? Jsou jednotky stejné?
- Neopakují se identifikátory? Nejsou důležité hodnoty prázdné?
- Je známé celé období, nebo nám část dat chybí?
- Sedí ručně ověřený součet a počet pro malý výběr?
- Odpovídají popisky, filtry i prázdné stavy novému významu dat?